

SQLmap & John the Ripper

Study Notes — Wednesday

1. Goal of SQLmap

SQLmap is **not** a magic hacking tool. It is a program that automates the SQL injection methodology previously performed manually in the PortSwigger labs.

Instead of manually writing SQL injection payloads, SQLmap generates payloads automatically, analyzes the server's responses, identifies vulnerabilities, and extracts database information.

2. Manual SQLi vs SQLmap

Previously, SQL injection was performed manually by:

- Adding ' to test SQL injection
- Using **UNION SELECT**
- Counting the number of columns
- Finding tables
- Finding columns
- Extracting usernames and passwords

SQLmap automates all of these steps.

Manual SQLi	SQLmap
Test SQL injection	-u
Find databases	--dbs
Select database	-D
Find tables	--tables
Select table	-T
Find columns	--columns
Extract data	--dump

3. Verifying SQLmap

Command:

```
sqlmap --version
```

This verifies that SQLmap is installed and working.

4. Basic SQLmap Command

```
sqlmap -u "http://example.com/product?id=1"
```

Meaning:

- **sqlmap** → Run SQLmap
- **-u** → Target this URL
- The URL is the webpage SQLmap will test

5. Why SQLmap Targets Parameters

Example:

```
http://example.com/product?id=1
```

SQLmap targets **id=1**, NOT **product**.

Reason:

- **product** is simply the webpage/path
- **id** is user-controlled input
- SQL injection occurs through user input

6. SQLmap Methodology

SQLmap does **not** immediately dump databases. It first:

- Sends the request
- Tests whether the parameter is injectable
- Determines which SQL injection techniques work
- Identifies the DBMS
- Begins enumeration

7. How SQLmap Detects SQL Injection

SQLmap sends many payloads, for example:

```
'  
AND 1=1  
AND 1=2
```

It compares the application's responses. If the responses change in meaningful ways, SQLmap concludes that the parameter is injectable — exactly what was previously done manually.

8. SQLmap Testing Techniques

Boolean-based Blind

SQLmap sends TRUE/FALSE conditions:

```
AND 1=1  
AND 1=2
```

If the page behaves differently, SQLmap identifies Boolean-based Blind SQL Injection.

Error-based

SQLmap intentionally causes database errors. If error messages reveal information, SQLmap uses them to extract database information.

Time-based Blind

Example payload:

```
SLEEP(5)
```

If the page waits approximately 5 seconds before responding, SQLmap knows the SQL query executed.

UNION Query

SQLmap automatically:

- Determines the number of columns
- Determines compatible data types
- Builds a valid UNION SELECT statement

This automates the manual process of repeatedly using:

```
UNION SELECT NULL,NULL
```

until the correct number of columns was found.

9. SQLmap Detection Results

SQLmap discovered:

- Boolean-based Blind SQL Injection
- Error-based SQL Injection
- Time-based Blind SQL Injection
- UNION-based SQL Injection

It also identified:

- **Database:** MySQL
- **Operating System:** Ubuntu Linux
- **Web Server:** Apache

10. Login Failure While Using SQLmap

Initially SQLmap failed. Reason: SQLmap requested

```
/dvwa/vulnerabilities/sqli/
```

without authentication. The server checked whether a valid session existed — the answer was **No** — so the server responded with a **302 Redirect** to **/login.php**. Therefore SQLmap never reached the vulnerable SQL Injection page.

11. Why Copying the Cookie Worked

I had already logged into DVWA through Firefox. The server created a session and sent cookies such as:

```
PHPSESSID=...  
security=low
```

Firefox automatically sends these cookies with every request. SQLmap initially did not. When the cookie was added:

```
--cookie="PHPSESSID=...; security=low"
```

SQLmap began sending the same cookies. The server checked whether the session was already authenticated — the answer was **Yes** — so SQLmap was allowed to access **/vulnerabilities/sqli/** instead of being redirected to **/login.php**.

12. Authentication vs Cookies

Situation 1 — No user has logged in

The only way to obtain a session is by sending a valid username and password. The server then creates a session.

Situation 2 — A valid session already exists

There is no need to send the username and password again. Sending the session cookie is enough because it proves the client is already authenticated.

The cookie acts like a hotel room key — the username/password are used only to obtain the key.

13. SQLmap Enumeration Workflow

List Databases

```
--dbs
```

Meaning: list every database available on the DBMS. Result included:

- dvwa
- information_schema
- mysql
- metasploit
- owasp10
- tikiwiki
- tikiwiki195

information_schema stores metadata such as databases, tables, and columns — SQLmap simply automates querying it.

Select Database

```
-D dvwa
```

Meaning: use only the DVWA database for future operations.

List Tables

```
--tables
```

Result:

- guestbook
- users

Select Table

```
-T users
```

Meaning: work only with the users table.

List Columns

```
--columns
```

Result:

- user
- avatar
- first_name
- last_name

- password
- user_id

Dump Data

```
--dump
```

Meaning: extract every row from the selected table. Result — SQLmap extracted:

- Usernames
- Password hashes
- First names
- Last names
- Avatars
- User IDs

14. SQLmap Sessions

After the first scan, SQLmap saved everything. Later commands displayed:

```
resuming back-end DBMS
```

Meaning: SQLmap remembered the injection point, database type, and successful payloads. Therefore it skipped repeating all detection tests — greatly speeding up later commands.

15. SQLmap Workflow Summary

```
Test URL
↓
Find SQL Injection
↓
Determine DBMS
↓
List Databases
↓
Select Database
↓
List Tables
↓
Select Table
↓
List Columns
↓
Dump Data
```

16. Password Hashes

SQLmap extracted password hashes. A hash is **not** plaintext. Example: the password

```
password
```

may become, using MD5:

```
5f4dcc3b5aa765d61d8327deb882cf99
```

17. John the Ripper

SQLmap retrieves hashes. John the Ripper attempts to recover the original passwords from those hashes. These are two completely different tools.

Creating a Hash File

Created **hashes.txt** containing:

```
5f4dcc3b5aa765d61d8327deb882cf99
```

First John Attempt Failed

```
john hashes.txt
```

John incorrectly assumed the hash type was **LM** (a Windows password hashing algorithm). The actual hash was not LM. Therefore John compared LM(candidate) against the MD5 hash, and no matches were found.

Why --format=Raw-MD5 Worked

```
john --format=Raw-MD5 hashes.txt
```

This forced John to treat the hash as MD5. Instead of calculating LM hashes, John calculated MD5 hashes for each candidate password. Eventually it generated **password**, computed MD5(password), and got:

```
5f4dcc3b5aa765d61d8327deb882cf99
```

The hashes matched — therefore the original password was recovered:

```
password
```

18. Important Concept About Hashes

The same password produces completely different hashes depending on the algorithm used. Example — password: **password**, across algorithms:

- MD5
- SHA-1
- SHA-256
- NTLM
- LM

Each algorithm produces a completely different hash. Therefore John must know which algorithm created the hash — otherwise it compares the wrong type of hashes and cannot crack them.

19. Does John Decrypt Hashes?

No. John never decrypts hashes. Instead it repeatedly performs:

```
Candidate password
↓
Apply hashing algorithm
↓
Generate candidate hash
↓
Compare with target hash
↓
Match?
```

If the hashes are identical, John has found the original password.

20. Complete Attack Workflow Learned Today

```
Find SQL Injection
↓
Verify vulnerability
↓
Determine DBMS
↓
Enumerate databases
↓
Enumerate tables
↓
Enumerate columns
↓
Dump password hashes
↓
Use John the Ripper
↓
Recover plaintext password
```

21. Key Takeaways

- 1. SQLmap automates manual SQL injection methodology.
- 2. SQLmap first detects SQL injection before performing enumeration.
- 3. Authentication is required for authenticated pages; session cookies allow SQLmap to reuse an existing logged-in session.
- 4. Cookies identify an already authenticated session, eliminating the need to send the username and password again.
- 5. SQLmap stores scan sessions and resumes them later to avoid repeating detection.
- 6. SQLmap extracts hashes; John the Ripper attempts to recover the original passwords.
- 7. Password hashes depend entirely on the hashing algorithm used.
- 8. John must know the correct hash format; otherwise it compares hashes generated by the wrong algorithm.
- 9. John does not decrypt hashes — it generates hashes from candidate passwords and compares them until a match is found.